AMOS: A Memory Operating Layer for Autonomous Data Analysis

Elton Chang

Abstract

Continuous large-scale data analysis agents operate across warehouses, streams, catalogs, semantic layers, documents, notebooks, and prior investigations. Their reliability depends not only on model reasoning, but on continuously managing the state that makes each computation valid: schema and metric versions, snapshots or stream offsets, freshness, permissions, feedback, execution records, and provenance. Prompt context, RAG, and text-to-SQL address parts of this problem but do not provide a durable operating abstraction for analytical state.

AMOS is an internally deployed analyst system that connects to company data and tools, answers business questions, performs verified analysis, and produces graphs, reports, and presentation slides. This paper studies its memory and control kernel: the layer between the included analyst agent and the systems it uses. The kernel separates bounded model-visible context from persistent, typed, versioned memory; filters permissions before context construction; reconciles temporal and authority conflicts; verifies computations against current schema and semantic state; and attaches claim-level provenance and replay metadata after execution. Raw data remains in existing data platforms while AMOS manages governed metadata, references, and analytical history through a stable runtime protocol.

We formalize the memory operating layer, implement a reference system, and evaluate its correctness, auditability, and scaling behavior. AMOS matches all outcomes in a 12-task invariant benchmark, compared with 6 of 12 for the strongest local baseline. Across payment failure, subscription churn, and warehouse quality, it satisfies all twelve seeded capability-contract variants per domain in three deterministic repetitions. Component ablations isolate the contribution of verification, permission filtering, and provenance. Indexed studies cover 1,000,001 memory objects and 1,000,000 provenance edges, while a governed BM25, MiniLM/HNSW, and hybrid comparison quantifies retrieval quality and latency through 100k distractors with zero restricted-item or superseded-item leaks. The results establish a concrete systems foundation for continuous large-scale data analysis agents whose context, computation, claims, and corrections remain governable over time.

1 Introduction

Language-model agents increasingly perform continuous data analysis: they translate questions into SQL, execute notebooks, inspect documents, monitor dashboards, and generate reports. At organizational scale, each task crosses changing data and metadata systems, and fluent output can hide state errors such as stale metrics, wrong schemas, incomplete late data, unauthorized evidence, or superseded analyses. The central systems problem is how an agent maintains the valid analytical state for every computation as workloads, users, and source systems evolve.

Continuous large-scale analysis is stateful. A request such as "why did anomaly rate increase in the last six hours?" may require an approved metric, stream offsets and watermarks, an active schema, ingestion delays, prior incidents, quality checks, and the query or chart supporting each claim. Existing data systems provide pieces of this state [10, 11, 12, 13, 15, 14]; an analytical agent still needs an operating layer that selects valid context, resolves conflicts, preserves corrections, and ties outputs back to execution evidence across tasks.

Existing LLM techniques are building blocks, not a full memory abstraction. RAG exposes external text [2]; long-context models remain position-sensitive [3]; tool-using agents interleave reasoning and actions [4, 5]; text-to-SQL benchmarks measure schema-grounded queries [6, 7, 8]. None guarantees temporally valid, permission-filtered, conflict-aware context linked to reproducible execution traces.

AMOS uses that memory operating layer inside a complete analyst product. The included analyst agent sits above it; warehouses, stream processors, semantic layers, catalogs, documents, execution tools, and lineage systems sit below it. These systems provide data, definitions, or traces; the kernel gives the agent one runtime contract for deciding which versioned and permissioned state enters a bounded working set, verifying computation against that state, recording claim support, and carrying reviewed corrections into later tasks. Product renderers then turn verified results into graphs, reports, presentation slides, dashboards, and spreadsheets.

System boundary. AMOS manages analytical state and references rather than copying unrestricted raw data into model memory. It targets controllable validity, governance, and auditability: analysts retain responsibility for causal and high-impact decisions, while every important claim remains inspectable through its data state, computation, definitions, memory versions, and verification record.

Contributions. The paper contributes:

1. a systems abstraction for a memory operating layer that connects continuous analysis agents to changing data, semantic, governance, and execution state;
2. a typed hierarchy and runtime protocol for bounded retrieval, supersession, reconciliation, verification, citation, feedback, and replay;
3. claim-level provenance linking outputs to data state, code, definitions, memory versions, permissions, and verification status;
4. a reference implementation and reproducible evaluation spanning invariant coverage, three analytical domains, component ablations, retrieval engines, concurrency, and million-object local scale.

Research questions. The evaluation is organized around four questions:

RQ1. **Operating-layer correctness:** does the runtime accept valid analytical computations and reject schema, metric, permission, freshness, and safety violations?

RQ2. **Persistent contract enforcement:** which guarantees require a memory operating layer rather than prompt-only, retrieval-only, semantic, or lineage policies?

RQ3. **Auditability cost:** what latency, storage, and replay overhead is added by claim-level provenance and governed state management?

RQ4. **Large-scale behavior:** how do indexed memory, provenance, retrieval quality, concurrency, and updates behave as state grows from thousands to millions of objects?

2 Background and Related Work

2.1 Memory, Context, and Operating Layers for LLM Agents

LLMs operate under finite context windows. Long-context models help, but they still require context selection, ordering, updating, and validation; models may underuse information placed in the middle of long prompts [3]. MemGPT treats the prompt as constrained main memory and external stores as archival memory [1]. Generative Agents stores observations and reflections for later planning, while Reflexion persists verbal feedback across trials [21, 22]. Surveys organize agent memory by sources, representations, operations, and evaluation strategies [23]. AMOS extends this line from memory as stored content to memory as an operating layer: a governed interface over schemas, metrics, snapshots, stream offsets, code, feedback, permissions, and provenance that mediates every analytical task.

2.2 Retrieval, Tool Use, and Text-to-SQL

RAG retrieves non-parametric evidence [2]; ReAct and Toolformer show how agents interleave reasoning with tool calls [4, 5]; Spider, BIRD, Spider 2.0, and EntSQL evaluate schema grounding, database interaction, enterprise metadata, dialects, codebase context, and business semantics [6, 7, 8, 9]. These directions motivate AMOS but leave a gap: query generation alone is insufficient when correctness depends on versioned memory, feedback retention, temporal validity, permissions, and provenance.

2.3 Streaming, Snapshots, and Reproducible Data States

Stream-processing systems formalize event time, watermarks, windows, state, and replay [10, 11, 12]; lakehouse systems provide transactions and time-travel snapshots [13]. AMOS does not reimplement these mechanisms. It

Runtime capability	RAG	Semantic layer	Catalog/ lineage	Long context	AMOS
Task evidence retrieval	**Yes**	Partly	Partly	**Yes**	**Yes**
Metric version by interval	Partly	**Yes**	Partly	–	**Yes**
SQL/schema/policy check	–	Partly	Partly	–	**Yes**
Pre-context permission filter	Optional	Partly	Partly	–	**Yes**
Authority/time reconciliation	–	Partly	Partly	–	**Yes**
Claim-level support links	–	–	Job-level	–	**Yes**
Durable reviewer feedback	Optional	–	–	–	**Yes**
Replay with memory/verifier state	–	–	Job-level	–	**Yes**

Table 1: Conceptual comparison with adjacent system classes. – denotes no typical standalone support; "Partly" denotes support that usually requires additional integration or policy. AMOS is intended to compose with these systems rather than replace them.

records references to the event-time window, watermark, snapshot version, query, and late-data policy used by an analytical claim.

2.4 Provenance, Lineage, Data Quality, and Security

PROV and OpenLineage define provenance and lineage vocabularies for entities, jobs, datasets, and runs [14, 15]. noWorkflow captures definition, deployment, and execution provenance from scripts, while YesWorkflow exposes prospective dataflow from lightweight annotations [24, 25]. Data-quality systems show the value of explicit validation constraints and drift checks [16, 17]. LLM-integrated applications add prompt-injection risk when external content is treated as instructions [18, 19, 20]. Recent runtime-governance work formalizes policies over partial agent execution paths rather than relying only on prompts or static access checks [26]. AMOS specializes this runtime-policy perspective to analytical state and claim support.

2.5 Positioning Against Existing System Classes

Table 1 summarizes the boundary between AMOS and adjacent systems. In deployment, AMOS should rely on them; its role is the runtime policy for selecting, reconciling, verifying, citing, and updating an analytical agent's working state.

The contribution is a compositional systems abstraction rather than a new retrieval algorithm or provenance vocabulary in isolation. The memory operating layer ties effective-time and authority-aware context selection to pre-execution verification, post-execution claim support, durable feedback, and replay through one executable contract.

3 Continuous Analysis as an Operating-Layer Problem

3.1 Continuous Large-Scale Analytical Task

A simplified text-to-SQL task maps a natural-language request q and schema context C to an executable query E:

$$q + C \to E. \tag{1}$$

For continuous large-scale analytics this abstraction is too small. At time t, a task depends on streams, snapshots, schemas, semantic definitions, documents, prior analyses, feedback, and permissions distributed across multiple systems:

$$\mathcal{T}_t = (q_t, S_{1:n}, H, C_t, D_t, A_{<t}, F_{<t}, P_t), \tag{2}$$

where $S_{1:n}$ are streams, H is snapshots, C_t is schema/catalog state, D_t is governed document context, $A_{<t}$ prior analyses, $F_{<t}$ feedback, and P_t the active access policy.

The agent must produce an artifact Y_t and a reproducible support record:

$$\mathcal{T}_t \to (Y_t, E_t, \mathcal{G}_t, R_t), \tag{3}$$

where E_t is executable computation, $\mathcal{G}_t$ is a provenance graph, and R_t is a replay package. Y_t may be prose, SQL, chart, notebook, dashboard note, or slide deck.

3.2 Analytical Memory State

AMOS manages memory state

$$\mathcal{M}_t = (M_t^a, M_t^s, M_t^c, M_t^m, M_t^d, M_t^p, M_t^f, M_t^g), \tag{4}$$

where M_t^a is active context, M_t^s is stream or snapshot state, M_t^c is schema/catalog memory, M_t^m is semantic memory, M_t^d is document memory, M_t^p is prior-analysis memory, M_t^f is feedback memory, and M_t^g is provenance memory.

The memory objective is to select a bounded working set $W_t \subset \mathcal{M}_t$ that is sufficient for the current task while satisfying freshness, access, authority, and provenance constraints:

$$W_t = \text{retrieve}(q_t, \mathcal{M}_t, P_t, B), \quad |W_t| \leq B, \tag{5}$$

where B is the active-context budget. The objective is not to maximize the amount of context, but to retrieve the smallest useful, current, permissioned, and source-backed context.

The operating-layer requirement is that model-visible context stays bounded as persistent analytical state grows. Let $N_t = |\mathcal{M}_t|$ be the number of memory objects and G_t the number of provenance edges. An agent should reason over $O(B)$ selected state while indexed storage, retrieval, reconciliation, update, and replay operations manage N_t and G_t outside the model context. This separation allows continuous analysis without making prompt length proportional to organizational history.

The formalism is operational: P_t is applied before context construction; C_t and M_t^m must match the SQL and metric definition in E_t; M_t^s identifies the data state behind numeric claims; $F_{<t}$ persists reviewed corrections; and $\mathcal{G}_t$ and R_t make artifacts inspectable. The evaluation tests these invariants in a controlled setting.

3.3 Failure Modes

AMOS targets system-state failures rather than hallucination alone: stale definitions, schema drift, late data, lost feedback, unsupported claims, unauthorized context, and prompt injection through retrieved content. Appendix Table A1 lists the representative failures used to shape the benchmark.

4 The AMOS Memory Operating Layer

4.1 Design Goals

AMOS has six design goals:

1. **Temporal correctness:** context must be valid for the requested time interval and data state.
2. **Bounded active context:** the agent should see only the relevant working set, not the entire organizational memory.
3. **Permission awareness:** memory retrieval and tool execution must respect user and task permissions.
4. **Provenance by default:** important claims should be tied to supporting data, code, definitions, and memory versions.
5. **Human-correctable memory:** reviewer feedback should update memory so repeated mistakes decrease over time.
6. **Scale separation:** persistent state and provenance may grow independently while model-visible context remains bounded and backend implementations remain replaceable.

4.2 Composition with the Data Stack

AMOS does not replace warehouses, lakehouses, stream processors, semantic layers, BI tools, catalogs, or lineage systems. It provides the memory and control interface that connects them to analysis agents. Raw data stays in governed source systems; AMOS maintains typed state, external references, execution records, and provenance so that changes in one source can invalidate, reconcile, or replay dependent analytical work.

4.3 Architecture

Figure 1 gives the system view. The analytical agent interacts with AMOS through a stable operating-layer contract. AMOS mediates retrieval, tool calls, verification, memory writes, provenance recording, and replay instead of directly injecting an unbounded collection of retrieved material into the prompt.

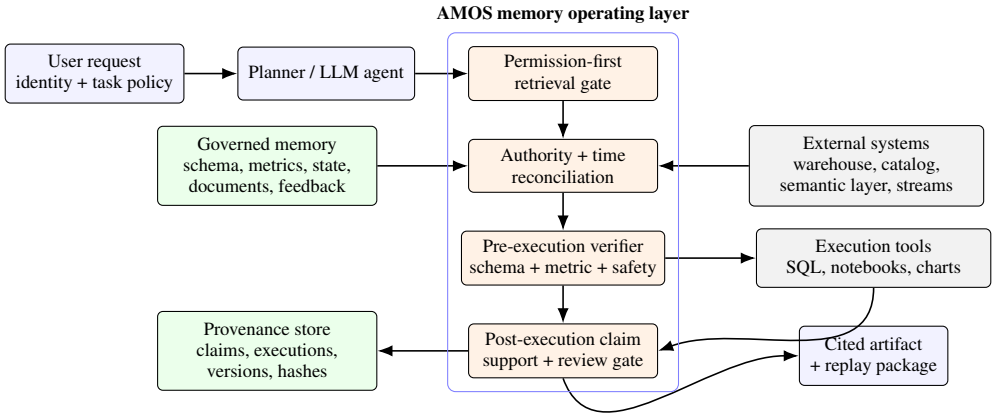

Figure 1: AMOS as a memory operating layer. A bounded model-visible working set sits above persistent governed memory; permission filtering and reconciliation precede execution; verification and claim-support gates mediate tools and publication; provenance feeds later retrieval and replay.

A minimal implementation requires bounded active context, persistent schema and semantic state, provenance memory, retrieval, verification, citation, and replay. Stream state, documents, prior analyses, and reviewer feedback extend the same operating interface without changing the agent-facing protocol.

4.4 Memory Tiers

AMOS separates active context from persistent tiers for stream or snapshot state, schema/catalog metadata, semantic definitions, documents, prior analyses, feedback, and provenance; Appendix Table A2 gives the detailed map.

5 Memory Model and Operating Primitives

5.1 Memory Object

A memory item is a typed object, not an arbitrary text chunk:

```
MemoryObject {
  id: stable identifier
  type: active_context | stream_state | schema | semantic_definition
        | document | prior_analysis | feedback | provenance
        | permission_policy
  content: structured metadata or an external-object reference
  summary: compact model-readable representation
  source: producing system or actor identifier
  authority: owner_approved | reviewer_approved | user_note
             | system_observed | model_hypothesis | untrusted_external
  effective_start, effective_end: interval described by the item
  transaction_time: time at which AMOS recorded the item
  permissions: required permission labels
```

```
  sensitivity: classification label
  version: source or policy version
  status: active | superseded | rejected | pending_review
  supersedes: identifiers replaced by this object
  provenance_ref: link to source object or producing computation
}
```

The split between `effective_time` and `transaction_time` matters: a document may be recorded today but describe last quarter, and a reviewed metric may apply only to future reports.

5.2 Runtime Invariants and Failure Semantics

The proposed contract is defined by five publication invariants. Let W_t be the model-visible working set, E_t the executable computation, Y_t the artifact, and P_t the user's policy:

$$I_1 : \forall i \in W_t,\ permitted(i, P_t), \tag{6}$$

$$I_2 : \forall i \in W_t,\ effective(i, t) \land \neg superseded(i, t), \tag{7}$$

$$I_3 : execute(E_t) \Rightarrow schemaValid(E_t, W_t) \land metricValid(E_t, W_t) \land readOnly(E_t), \tag{8}$$

$$I_4 : publish(Y_t) \Rightarrow \forall c \in importantClaims(Y_t),\ supported(c, \mathcal{G}_t), \tag{9}$$

$$I_5 : replayable(Y_t) \Rightarrow complete(R_t, W_t, E_t, \mathcal{G}_t). \tag{10}$$

I_1 is enforced before model context construction; I_2 during retrieval and reconciliation; I_3 before tool execution; and I_4–I_5 before publication. These implications define the operating-layer boundary: sources and causal interpretations retain their own review requirements, while state validity and claim support become executable runtime properties.

The runtime returns one of five outcome classes. `reject` blocks execution or publication after an unresolved safety or validity failure. `repair` proposes a modified computation that must pass the same checks from the beginning. `warning` permits a mechanically valid artifact while exposing incomplete freshness or other bounded uncertainty. `needs_review` keeps causal or operational claims non-final. `pass` means the configured structural checks succeeded.

The reference prototype uses snapshot consistency: each task records the memory IDs and external snapshot or offset references observed during the run. The deployment design pairs this record with atomic artifact/provenance publication, immutable source versions or leases, and invalidation when policy, schema, or metric state changes.

5.3 Core Operating Primitives

AMOS exposes system-level primitives rather than prompting conventions: retrieve, write, supersede, reconcile, compact, verify, cite, and replay. These primitives form the stable agent-facing API while storage, retrieval, execution, and provenance backends can evolve independently. Appendix Table A3 defines their contracts.

5.4 Runtime Protocol

The runtime protocol starts with policy construction so inaccessible memory never enters model-visible context. AMOS retrieves candidates, reconciles conflicts, builds a bounded working set, verifies tool calls, records execution, cites claims, and writes durable feedback only after verification gates run.

```
AMOS-RUN(q, user, policy, budget, provenance_level):
  task <- parse(q)
  candidates <- retrieve(task, policy, budget)
  working_set, conflicts <- reconcile(candidates)
  if unresolved(conflicts): return needs_review(conflicts)

  plan <- agent.plan(task, working_set)
  for step in plan.tool_steps:
      verify(step.sql_or_code, working_set, policy)
      result <- execute(step)
```

```
      record_execution(result)

  artifact <- agent.compose(task, working_set, results)
  claims <- extract_claims(artifact)
  for claim in claims:
      cite(claim, results, working_set, provenance_level)

  verification <- verify(artifact)
  replay_package <- package(task, plan, working_set, results,
                            verification)
  write(artifact, claims, provenance, replay_package)
  return artifact, verification, replay_package
```

The ordering is the system contract: permission filtering precedes context injection, reconciliation precedes planning, verification precedes execution, provenance precedes publication, and memory writes preserve authority and review status. Deployments may replace individual steps with learned retrievers or richer verifiers.

5.5 Retrieval and Reconciliation

Retrieval combines semantic relevance with system constraints. The correctness-first prototype favors authority, effective time, supersession status, field-level matches, and permission filtering over raw lexical recall:

$$score(i, q, t, u) = \alpha m(i, q) + \beta a(i) + \gamma f(i, t) + \delta v(i) + \eta s(i) - \lambda c(i, u), \tag{11}$$

where m is field-aware match quality, a authority, f freshness, v verification/review status, s active non-superseded status, and c visible sensitivity risk. Access violations are filtered before scoring.

```
GOVERNED-RETRIEVE(q, user, time_range, budget):
  candidates <- lexical_candidates(q, candidate_budget)
  visible <- []
  for item in candidates:
      if not permitted(user, item.permissions):
          record_filtered(item)
          continue
      visible.append(item)

  scored <- []
  for item in visible:
      score <- field_match(q, item.id, item.type,
                           item.summary, item.content)
      score += authority_score(item.source, item.authority)
      score += freshness_score(item.effective_time, time_range)
      score += verification_score(item.status, item.review_state)
      score += supersession_score(item.status, item.supersedes)
      score -= sensitivity_penalty(item, user)
      scored.append((score, item))

  ranked <- sort_desc(scored)
  warnings <- ambiguity_warnings(ranked, q, time_range)
  return top_k(ranked, budget), warnings
```

Ambiguity handling is part of the success criterion: when similarly named active items plausibly match, AMOS surfaces a warning and keeps the approved task-relevant item near the top rather than silently accepting the first lexical match.

Conflict resolution follows a partial order, not blind recency. A default authority order is:

$$owner\_approved > reviewer\_approved > user\_note > model\_hypothesis. \tag{12}$$

Effective time constrains this order: a newer model hypothesis cannot override an older owner-approved definition that remains effective. Conflicting owner-approved definitions for the same interval are surfaced, not silently resolved.

5.6 Stream-Aware but Not Stream-Only

AMOS is stream-aware, not stream-only. In batch settings, M_t^s stores snapshot identifiers, table versions, query timestamps, partition ranges, and data-quality status rather than offsets and watermarks; the other tiers remain unchanged.

6 Reference Implementation

We implemented the memory operating layer as a deterministic reference system: SQL is template-selected, retrieved memory is typed, verification runs before execution, and artifacts are replayable from saved metadata. This isolates operating-layer behavior from variation in natural-language SQL generation.

The implementation uses SQLite FTS5/BM25 candidate generation followed by permission, temporal, authority, reconciliation, and reranking checks. Separate MiniLM/HNSW and reciprocal-rank-fusion paths evaluate how candidate generation affects the same governed protocol. The agent-facing operations remain unchanged across these backends.

6.1 System Components

The artifact uses SQLite for one-command reproduction. The schema separates memory objects, artifacts, claims, provenance edges, replay packages, and audit logs, with JSON payloads for versioned metadata. FTS5 triggers synchronize the lexical index with memory writes; source/target indexes support provenance lookup. Candidate-generation and storage backends change scale characteristics without changing the memory operating contract. Appendix Table A4 summarizes the components.

6.2 Legacy Synthetic Payment Evaluation Scenario

This synthetic scenario is a controlled systems fixture, not the AMOS product definition or intended customer specialization. The prototype seeds 14,400 payment events over twelve hours. Failures rise in the final six-hour event-time window and concentrate in Processor B / Visa transactions. The dataset includes excluded test accounts, a deployment before the spike, late arrivals represented by watermark lag, and a schema migration from `failure_reason` to `error_code`. Seeded memory includes the approved `payment_failure_rate:v3` metric, current `payment_events:v2` schema, stream state, a restricted prior incident, deployment evidence, reviewer feedback against over-attribution, and an analyst policy.

For the main request, the controller retrieves the approved metric, current schema, stream state, deployment note, reviewer feedback, and policy, while filtering the restricted incident for a user without `sre`. The report states that failure rate rose from 2.2% to 7.4%; Processor B / Visa reached 15.8%. Deployment evidence remains review-gated rather than causal proof.

6.3 Replay Package

Each run writes a replay package with the request, plan, memory IDs, schema and metric versions, stream state, SQL, chart IDs, result hashes, and verifier status. Replay re-executes the saved SQL against DuckDB, regenerates the chart, and compares hashes, demonstrating that a continuous analytical artifact can carry the state required for audit and regeneration.

7 Provenance, Verification, and Security

7.1 Claim-Level Provenance

Artifact-level lineage is too coarse: one report may contain a correct chart, an outdated metric, and an unsupported causal explanation. AMOS records provenance at the claim level:

```
ClaimProvenance {
  claim_id: sentence, chart annotation, slide bullet, or table cell
  artifact_id: report, notebook, chart, dashboard, or deck
  support: [query_id, notebook_cell_id, chart_id,
            document_ref, memory_object_id]
  data_state: [snapshot_id | stream_offset_range,
               watermark, event_time_window]
  semantic_state: [metric_definition_id, schema_version,
                   unit_definition]
  execution_state: [engine, runtime, parameters,
                    code_hash, result_hash]
  verification_state: [passed_checks, warnings,
                       unresolved_conflicts]
}
```

This structure lets reviewers identify claims using superseded metrics, restricted sources, late-data-sensitive outputs, or expired replay state.

Figure 2 shows one concrete support graph. Edges are typed rather than generic citations: the numeric claim is computed by a query and chart, governed by metric and schema versions, scoped to a recorded data state, and checked by a verifier whose result enters the replay package.

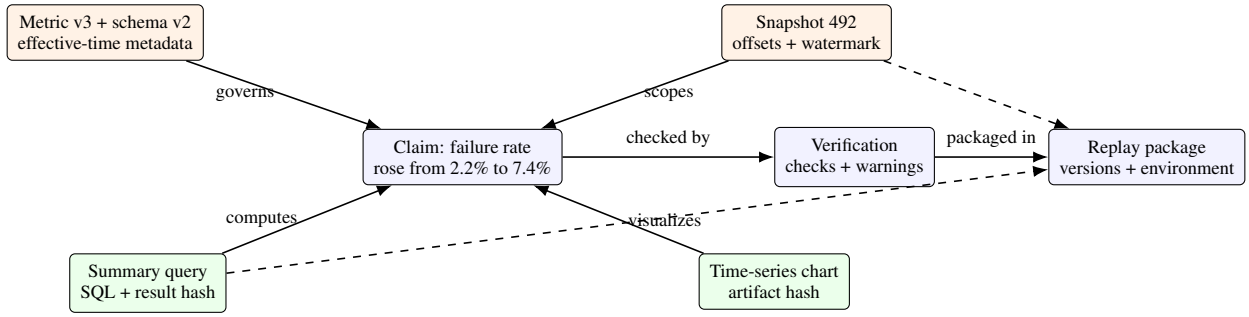

Figure 2: Worked claim-level provenance graph. Support edges distinguish computation, semantic governance, data state, verification, and replay rather than attaching one artifact-level source list.

The prototype uses claim-type-specific support plans. Numeric rate claims cite summary/time-series queries and charts; concentration claims cite concentration queries; deployment and dashboard claims cite numeric, document, and approved-feedback evidence and remain review-gated. Coverage is complete only when support categories match the claim type.

7.2 Provenance Levels

Full replay is expensive and unnecessary for low-risk exploration. AMOS therefore supports configurable provenance levels: high-risk reporting should use claim-level provenance or full replay; exploration may use artifact- or chart/query-level records. Appendix Table A5 lists the levels.

7.3 Concrete Verifier

Verification is structural and semantic, not a proof of truth. The prototype verifier rejects unsafe or stale computations before execution and downgrades unsupported interpretations after execution. It runs six checks.

SQL safety. The parser must accept the query, and the root statement must be read-only. The verifier rejects writes, DDL, export/copy commands, and blocked raw columns such as `customer_email`, `payment_token`, or `raw_payload`.

Schema validity. SQL table and column references are compared with schema memory. For the current payment schema, `error_code` is valid and superseded `failure_reason` is rejected before execution.

Metric compliance. The verifier checks the parsed SQL AST against the approved metric definition. For v3 of the payment-failure metric, the numerator must count `status = 'failure'` inside an aggregate, the denominator must include unfiltered attempts with `COUNT(*)`, and row filters must enforce production traffic, test-account exclusion, and an `event_time` window. Equivalent predicates pass; comments or string literals do not.

Freshness and temporal state. Artifacts must attach stream or snapshot state. The payment run records snapshot `payment_events_snapshot_492`, interval 2026-07-07T14:00:00Z–20:00:00Z, offsets 125000–148999, watermark 19:58:30Z, and a 15-minute late-data policy. The 90-second watermark lag emits a warning.

Permission safety. Retrieved memory is permission-checked before active-context construction. A user with `analytics,payments` receives aggregate payment memory and deployment evidence, not the prior incident requiring `sre`.

Provenance completeness and review obligations. At level 3, numeric, causal, and recommendation claims must link to claim-appropriate queries, charts, metric/schema memory, data state, execution hashes, and verification status. Causal and operational claims also require document or approved-feedback evidence and remain `requires_review` unless backed by an approved causal design.

The core controller emits explicit claim records from structured report templates. An additional extractor handles prose from reports, notebooks, and slides and is evaluated separately in Section 8.9. Verification establishes compliance with encoded analytical and governance rules; causal interpretations remain review-gated.

7.4 Security Model

AMOS treats retrieved content as untrusted. The security design uses five controls:

1. **Permission-first retrieval:** inaccessible memory is filtered before model context construction.
2. **Instruction/data separation:** retrieved text is labeled as evidence, not policy.
3. **Tool sandboxing:** execution tools run with least-privilege credentials and auditable parameters.
4. **Memory-write gates:** model-inferred memory remains low authority until reviewed.
5. **Sensitive provenance handling:** provenance views are redacted according to user role and task need.

The operating layer composes these controls with existing access control, secret isolation, execution sandboxes, audit logs, and organization-specific approval policies.

The seeded security suite covers indirect prompt injection, malicious prior analyses, malicious reviewer feedback, safe feedback rendering, provenance-link leakage, and cross-user filtering. AMOS passes all six after adding authority-aware feedback ranking and separating low-authority evidence from applied reviewer guidance.

8 Evaluation of Operating-Layer Correctness and Scale

The evaluation tests whether the AMOS memory operating layer enforces its contract across changing analytical state, preserves auditability, and maintains bounded agent context as persistent memory and provenance grow.

8.1 Benchmark Workload and Baselines

The evaluation has two layers. The **12-task invariant benchmark** decomposes the payment-failure scenario into spike analysis, metric drift, schema drift, late data, permission conflict, feedback retention, replay, stale-document reconciliation, prompt injection, memory poisoning, causal review, and distractor retrieval. The core dataset has 14,400 synthetic payment events, a six-hour spike, a schema migration, a metric-definition change, a 90-second watermark

Task family	Perturbation	Expected evidence or decision	Gate
Metric/schema drift	Superseded metric or column is salient.	Uses metric v3/schema v2; rejects stale SQL before execution.	Verifier
Late data	Watermark trails the interval by 90 seconds.	Records offsets and watermark; emits freshness warning.	Stream-state check
Permission conflict	Relevant prior incident is restricted.	Excludes restricted memory from context and citations.	Retrieval gate
Feedback retention	Reviewer correction is written once.	Later task retrieves correction and avoids over-attribution.	Memory write/retrieve
Replay/provenance	Saved artifact is regenerated.	SQL hashes, chart metadata, memory IDs, and verification state match.	Replay engine
Adversarial memory	Injection text or low-authority metric is retrieved.	Treats text as data; approved memory prevails.	Reconciliation/write gate
Causal review	Deployment note precedes spike.	Marks causal and dashboard-update claims `needs_review`.	Claim verifier
Distractor retrieval	5,000 unrelated memories are added.	Approved metric remains rank 1.	Retriever

Table 2: Task protocol. The benchmark scores the expected system decision, including warnings, rejections, and `needs_review` outcomes, rather than rewarding only complete answers.

lag, one restricted prior incident, one deployment note, and reusable reviewer feedback. Retrieval probes add 5,000 distractor memory objects plus paraphrased, ambiguous, stale-similar, near-duplicate, and permission-dependent cases.

The **offline capability-contract evaluation** expands beyond the invariant suite. It runs fourteen local systems and adapters across payment failure, subscription churn, and warehouse quality, with twelve seeded variants and three deterministic repeats per domain. Variants inject dashboard pressure, wording paraphrases, late-data probes, prompt-injection documents, restricted-incident traps, schema-rename traps, and stale-metric traps. Full-contract pass requires analytical correctness, permission safety, the review boundary, claim provenance, and replay; metric-axis tables report partial capability separately.

On the invariant benchmark we compare five local baselines. **Metadata-filtered RAG** calls the local retriever with access-control and status filtering but lacks metric/schema verification and replay. **Semantic-layer agent** validates approved metrics but lacks schema, stream-state, provenance, and permission gates. **Catalog+lineage+dbt agent** uses schema, metric, stream-state, and permission checks, but lacks claim-level citation and feedback memory. **Strong long context** serializes memory broadly without pre-context permission filtering, authority reconciliation, or replay. **Structured tool-using LLM agent** remains a deterministic offline simulation in the invariant table for repeatability; the live-model pilot is reported separately.

The capability analysis adds the same families plus two **strong local baselines** (manual policy-prompt agent and permission-filtered RAG) that receive current metric/schema context and permission filtering; three **OSS-shaped adapters** that load exported-interface fixtures (SQLite FTS5 RAG documents, MetricFlow/dbt-metrics JSON, and OpenLineage-shaped events); and three **component ablations** that disable the verifier, permission gate, or provenance recorder one at a time.

8.2 Task Protocol

Each task is an input request plus a seeded memory/data perturbation. The predefined success criterion is structural: required SQL predicates, memory versions, permission exclusions, claim-specific provenance links, and final status. For example, metric drift passes when the AST-backed verifier rejects or repairs a query that omits test-account exclusion; causal review passes when deployment and dashboard claims are marked `needs_review`. Table 2 summarizes the protocol.

Baselines receive the seeded metadata their capability would normally consume. They fail only when their local policy lacks an operation required by the criterion, such as durable feedback memory, claim-level replay, or pre-context instruction/data separation.

Reported AMOS invariant-benchmark runtime is averaged over three deterministic local runs. The extended harness adds local baseline policies, noisy retrieval variants, a 50-case invariant regression suite, a frozen verifier engineering corpus, a 1,000-example claim-extraction corpus, seeded security probes, 5,000-distractor retrieval, and concurrent retrieval measurements. Capability-contract executions use the deterministic offline live-agent provider and record per-run contracts, metric axes, failures, and provenance-overhead pairs.

System	Matches	Tasks	Rate	Mean runtime
AMOS	**12**	**12**	**1.000**	**0.268 s**
Catalog+lineage+dbt agent	6	12	0.500	n/a
Metadata-filtered RAG	3	12	0.250	n/a
Semantic-layer agent	3	12	0.250	n/a
Strong long context	3	12	0.250	n/a
Structured tool-using LLM	1	12	0.083	n/a

Table 3: Aggregate results over the 12-task benchmark. A match means the system produced the expected decision, including `warning`, `reject`, or `needs_review` where required. AMOS runtime is averaged over three local analysis runs; baseline timings are omitted because the baselines are component policies rather than comparable end-to-end systems.

System	Payment	Churn	Warehouse	Provenance / replay
AMOS	**12/12**	**12/12**	**12/12**	**1.0 / 1.0**
Strong baselines (policy prompt; RAG+perm)	0/12	0/12	0/12	0.0 / 0.0
OSS-shaped adapters (FTS5 RAG; metrics JSON; OpenLineage)	0/12	0/12	0/12	0.0 / 0.0
Simple RAG / semantic / catalog / long context / agent-only	0/12	0/12	0/12	0.0 / 0.0
AMOS no verifier	0/12	3/12	0/12	1.0 / 0.5–0.75
AMOS no permission gate	0/12	0/12	0/12	1.0 / 1.0
AMOS no provenance	0/12	0/12	0/12	0.0 / 1.0

Table 4: Offline capability-contract matches by seeded variant; each variant is repeated deterministically three times. A variant passes when all repeats satisfy task correctness, permission safety, review boundary, claim provenance, and replay.

8.3 Metrics

The primary outcomes are state correctness, bounded-context governance, auditability, replay, retrieval quality, update visibility, and indexed scaling behavior. Appendix Table A6 defines the metric set used by the harness.

8.4 Benchmark Results

Table 3 reports aggregate criterion matches. AMOS matches all 12 outcomes, including non-final warning, rejection, downgrade, and `needs_review` decisions. The strongest baseline, catalog+lineage+dbt, matches 6 of 12: it handles schema, metric, late-data, permission, memory-poisoning, and retrieval-stress checks, but fails integrated spike analysis, feedback retention, replay, stale-document reconciliation, prompt-injection isolation, and causal-review obligations. The offline structured-LLM baseline passes only schema syntax.

AMOS returns a warning for the integrated report because the watermark lags the requested window and causal/dashboard claims require review. The artifact remains replayable and cited, while tentative causality is not presented as final.

8.5 Task-Level Results

Appendix Table A7 gives task-level results for selected baselines. The AMOS status is the observed system decision, not just a binary success label.

8.6 Multi-Domain Capability-Contract Evaluation

Table 4 summarizes full-contract outcomes. Across payment failure, subscription churn, and warehouse quality, AMOS passes all twelve seeded variants in all three deterministic repeats per domain. The all-or-nothing result is reported with metric-axis scores so analytical capability can be distinguished from operating-layer provenance and replay coverage.

The metric-axis breakdown clarifies the comparator story (Appendix Table A10). In the payment domain, both strong baselines and the OSS semantic/catalog adapters score 1.0 on task, SQL, metric, schema, permission, and review axes, yet 0.0 on provenance and replay. The OSS FTS5 RAG adapter reaches 0.75 on the analytical axes with permission safety 1.0. Catalog and semantic legacy baselines can ground SQL and metrics without satisfying the full

memory contract. Ablations confirm complementary failure modes: disabling the permission gate collapses permission safety while preserving provenance and replay; disabling provenance recording preserves analytical axes and replay but fails claim support; disabling the verifier damages SQL/metric/schema compliance and reduces replay success.

Matched provenance-overhead executions hold provider, prompts, SQL/tool path, verifier, and replay constant while disabling claim recording. After repeats are collapsed by seeded variant, the payment-domain mean latency delta is about 0.047 s and the measured raw-evidence delta is about 198 bytes.

Scenario-pack manifests cover three domains with capability-evaluation and live-agent adapters; 120 generated cross-domain task variants pass manifest-contract checks. The scored tables use the twelve executed variants per domain.

8.7 Ablations and Large-Scale Memory Behavior

Appendix Table A8 reports ablations on the 12-task invariant benchmark. Removing the verifier drops five tasks; removing stream/snapshot memory or semantic memory drops three each.

We replaced full-store scans with FTS5/BM25 candidate generation and reran one fixed local protocol at 10,001, 100,001, and 1,000,001 total memories, paired with 10,000, 100,000, and 1,000,000 provenance edges. The approved metric remains rank 1 in 30/30, 30/30, and 20/20 serial retrievals. Serial p50 latency rises from 0.045 s to 0.145 s and 4.997 s; p95 rises from 0.047 s to 0.152 s and 8.488 s. At eight readers, p95 is 0.411 s, 0.717 s, and 4.548 s. All runs keep the FTS row count synchronized, observe permission revocation and metric supersession, and complete their mixed read/write workload without recorded errors. The million-scale database occupies about 1.50 GB after all probes. Figure 3 shows the latency curve.

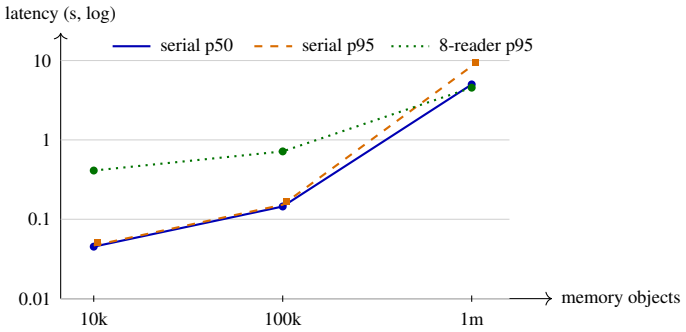

Figure 3: Single-machine indexed memory retrieval. The model-visible working set remains bounded while persistent state grows from 10k to one million objects; rank-1 correctness holds at every scale and the curve quantifies backend tail latency.

Provenance insertion throughput is about 109k edges/s at 10k edges, 73k at 100k, and 44k at one million. Indexed target-query p95 is 0.0009 s, 0.0027 s, and 0.273 s, respectively. Together with memory retrieval, these measurements demonstrate the operating-layer separation: analytical history scales outside the prompt, while the agent consumes a fixed-budget working set. Candidate generation and million-object tail latency become backend optimization targets rather than changes to the agent protocol.

8.8 Governed Lexical, Vector, and Hybrid Retrieval

We separately compare three candidate-generation paths while holding the post-retrieval operating contract fixed: SQLite FTS5/BM25 plus governed reranking; cosine HNSW over pinned MiniLM-L6-v2 embeddings (revision prefix `1110a243fdf4`, $M = 16$, `ef_construction=200`, search `ef=128`); and reciprocal-rank fusion ($k = 60$). The bundle records the full model identifier and revision hash. The controlled workload contains twelve target metrics, 24 lexical or semantic-paraphrase queries, restricted and superseded near-duplicates, and templated distractors. Table 5 reports the frozen results at 1k, 10k, and 100k distractors.

Distr.	BM25 top-1	BM25 p95	HNSW top-1	HNSW p95	Hybrid R@5	Hybrid p95
1k	0.667	0.267 s	0.917	0.013 s	0.917	0.283 s
10k	0.667	2.036 s	0.708	0.016 s	0.875	1.860 s
100k	0.667	17.326 s	0.208	0.030 s	0.833	16.746 s

Table 5: Governed retrieval-engine comparison. HNSW query latency stays low, but fixed-configuration relevance collapses in duplicated semantic neighborhoods at 100k. Hybrid retrieval preserves broader recall while inheriting the lexical full-store fallback tail. Across all engines and scales, permission-leak and superseded-item-leak counts are zero.

MiniLM/HNSW is strongest at 1k but loses relevance as duplicated semantic neighborhoods crowd the fixed index; hybrid retrieval recovers recall while inheriting the lexical tail. Permission-revocation and metric-supersession probes pass at every archived scale. At 100k, SQLite/FTS5 seeding and indexing takes approximately 6.8 s and 70 MB, whereas embedding plus HNSW construction takes approximately 667 s and 169 MB. The result motivates adaptive candidate generation beneath the same operating layer: vector search for latency, lexical evidence for recall, and governance checks after fusion.

8.9 Extended Operating-Layer Experiments

The extended harness covers verifier regressions, retrieval ambiguity, invariant variants, scaled claim extraction, seeded security attacks, OSS-shaped adapter probes, and concurrency. Appendix Table A9 summarizes the results.

Noisy retrieval and security probes exposed failures in the earlier keyword-only retriever and report renderer. The revised retriever adds field-aware matching, authority-aware feedback ranking, supersession-aware scoring, and ambiguity warnings; the report generator separates approved feedback from low-authority evidence. Canonical, paraphrased, stale-similar, and permission-dependent probes rank the approved metric first; the ambiguous metric-name probe warns and keeps it in the top two.

Claim extraction is evaluated on a 1,000-example seed-controlled corpus spanning report, notebook, slide, ambiguous, subscription, and warehouse artifacts. A regex/keyword extractor reaches mean type precision 0.958, recall 1.0, and review-obligation recall 1.0.

The artifact also includes versioned intake contracts for independent holdout tasks and real claim annotations. Validators hash sealed references, enforce author/reviewer/annotator/adjudicator separation, prevent source-group leakage across splits, preserve pre-adjudication labels, verify exact annotation spans, and align task IDs before paired analysis. This creates a reproducible path from controlled operating-layer tests to independent external evaluation.

After replacing flat column-name checks with scoped SQL qualification, a frozen 16-case regression corpus accepts all six labeled-valid queries and rejects all ten labeled-invalid queries. Cases cover aliases, CTE outputs, conditional aggregates, stale and unknown columns, blocked columns, missing metric filters, wrong time fields, adversarial text, and writes.

The archived live feasibility pilot uses the authenticated OpenAI Codex CLI with model `gpt-5.6-sol`. A strict lexical grader marks 3 of 8 policy responses as passing, while all three corrected-verifier end-to-end tasks complete with the expected `warning` status for watermark lag and review-gated causal claims. One SQL-proposal call times out at 180 seconds; its raw trace is preserved and the isolated retry completes, yielding 3/3 end-to-end completions and zero unresolved provider failures.

8.10 Evaluation Scope and External Protocol

The reported results are controlled reference-system evidence: deterministic task variants measure contract enforcement, fixed system studies measure indexed state growth, and the live pilot checks end-to-end feasibility. We therefore report exact task counts and fixed-workload measurements rather than population estimates. A frozen external protocol extends this foundation with 150 independently authored tasks across three domains, multiple model families and prompt frames, double-annotated real artifacts, red-team testing, a reviewer audit study, and a production-faithful comparison system. Separation of task authors, annotators, adjudicators, and system developers is enforced by executable intake validators.

9 Discussion and Systems Roadmap

9.1 Why an Operating Layer?

RAG retrieves evidence; catalogs describe assets and governance metadata; semantic layers define metrics; lineage systems track datasets, jobs, and runs. AMOS does not replace these systems. It treats them as memory sources and execution references, while adding runtime control of the agent's evolving analytical working state.

The distinction is operational. RAG can retrieve a metric document without deciding whether its version is effective for the requested interval. A catalog can expose a column without ensuring that a report used the valid schema version. A semantic layer can define `payment_failure_rate` without attaching it to every textual claim. A lineage tool can record a table read without carrying a reviewer correction into the next task. AMOS supplies the shared layer above them: retrieve bounded context, check temporal validity and permissions, reconcile conflicts, execute tools, verify outputs, attach claim-level provenance, replay prior work, and persist corrections.

9.2 Large-Scale Deployment Model

The operating layer is risk-adaptive. Exploratory analysis can use artifact-level records, while executive or regulated reporting can require claim-level support and full replay. In the matched payment workload, claim recording adds about 0.047 s mean latency and 198 bytes to measured raw evidence. Persistent state is indexed outside the model, and the active-context budget remains fixed; compaction, retention, invalidation, and storage backends can therefore be tuned without changing the agent-facing API.

The integration path is direct: catalogs and semantic layers populate authoritative memory; warehouse snapshots and stream processors provide data-state references; execution engines provide code and result hashes; and lineage systems receive the resulting claim graph. The systems roadmap advances this composition through distributed metadata storage, adaptive hybrid retrieval, policy churn, concurrent execution, and connectors for dbt, OpenLineage, DataHub/OpenMetadata, warehouse snapshot APIs, and orchestration logs.

10 Conclusion

Continuous large-scale data analysis agents need more than stronger models, larger prompts, or better SQL generation. They need a memory operating layer that manages schemas, semantic definitions, stream or snapshot state, prior analyses, review feedback, permissions, execution history, and provenance as first-class resources. AMOS defines that abstraction and implements it as a bounded-context protocol over persistent, typed analytical state.

The central result is that continuous analysis is memory-bound as well as model-bound. Agents must know which context is valid, which definitions are authoritative, which data state was used, which claims are supported, and which corrections persist. AMOS turns these requirements into operating-layer invariants. It satisfies the full contract across the invariant benchmark and three analytical domains, preserves permission and supersession controls across retrieval engines, and records replayable claim support while indexed state reaches one million memory objects and one million provenance edges. This establishes a concrete systems foundation for agents that continuously analyze large and changing data environments without allowing prompt context to become the system of record.

A Detailed Tables

Failure mode	Risk	AMOS control
Temporal staleness	Answer uses an expired definition or data state.	Effective-time and supersession checks.
Schema drift	Query references a removed or renamed column.	Schema-version validation before execution.
Metric drift	A metric means different things across teams or dates.	Authoritative semantic memory with effective dates.
Late data	Result ignores events arriving after the first watermark.	Recorded offsets, watermarks, and late-data policy.
Feedback loss	Reviewer correction is not applied later.	Durable feedback memory and retrieval constraints.
Unsupported claim	Report states a cause without support.	Claim-level provenance and citation checks.
Unauthorized retrieval	Agent uses restricted data or documents.	Permission filtering before context injection and tool execution.
Prompt injection	Retrieved text issues malicious instructions.	Instruction/data separation, sandboxed tools, and write gates.

Table A1: Representative analytical-agent failure modes and the AMOS controls that make them inspectable.

Tier	Stores	Used for
Active context	Current task, selected definitions, schema, and recent tool results.	Model-visible working set.
Stream/snapshot state	Offsets, watermarks, event windows, snapshot IDs, late-data policy.	Temporal validity and replay.
Schema/catalog	Tables, columns, joins, owners, sensitivity labels, schema versions.	Query grounding and permissions.
Semantic definitions	Metrics, rules, units, dimensions, effective dates, owners.	Consistent meaning across analyses.
Documents	Runbooks, incident reports, requirements, design notes, summaries.	Source-backed context treated as evidence.
Prior analyses	Accepted SQL, notebooks, charts, explanations, dashboard notes.	Reuse work and preserve analytical history.
Feedback	Corrections, rejected assumptions, approvals, escalation notes.	Durable human review.
Provenance	Claim-query, query-data, artifact-memory, and version links.	Auditability, impact analysis, and replay.

Table A2: Typed memory tiers managed by AMOS.

Operation	Contract
`retrieve(q, policy)`	Return a bounded, permissioned, temporally valid working set.
`write(item)`	Persist memory with source, authority, time, permissions, and provenance.
`supersede(old,new)`	Replace an item while preserving lineage.
`reconcile(items)`	Resolve conflicts by authority, effective time, review status, and source type; surface unresolved conflicts.
`compact(items)`	Shrink memory while preserving provenance and validity metadata.
`verify(artifact)`	Check schema, metrics, freshness, permissions, execution status, and provenance coverage.
`cite(claim)`	Link a claim to supporting data, code, definitions, memory, and verification results.
`replay(artifact)`	Re-execute or audit using recorded snapshots, offsets, code, and memory versions.

Table A3: Core AMOS memory operations.

Role	Local implementation	Production role
Memory/provenance store	SQLite tables for memory, artifacts, claims, provenance edges, replay packages, and audit logs.	Durable governed state; production could use PostgreSQL/JSONB/pgvector.
SQL engine	DuckDB over synthetic payment events.	Read-only analytical execution and deterministic result-hash replay.
Notebook-style runner	Python controller for parse, retrieve, plan, verify, query, chart, report, cite, and package.	Ordered, testable execution similar to notebook cells.
LLM-agent harness	Offline provider plus OpenAI Responses and authenticated ephemeral Codex CLI transports.	Preserves raw prompts, outputs, model identity, latency, token totals, verifier failures, and repairs.
Verifier	Rule checks over SQL ASTs, schema, metrics, stream state, policy, and claims.	Rejects invalid computation and marks unsupported claims for review.
Artifact store	Filesystem Markdown, SQL, PNG, JSON provenance, and replay files.	Inspectable reports, charts, queries, and replay metadata.

Table A4: Reference implementation components and their corresponding deployment roles beneath the memory operating layer.

Level	Recorded support	Typical use
0	No durable provenance.	Disposable exploration.
1	Artifact-level inputs and tools.	Low-risk reports or notebooks.
2	Chart/query links to source computations.	Reused charts and SQL outputs.
3	Claim links to computations, definitions, data state, and memory versions.	High-risk reporting.
4	Full replay package: code, parameters, snapshots/offsets, memory versions, and environment metadata.	Audits and regulated workflows.

Table A5: Configurable provenance levels. Higher-risk artifacts should use claim-level provenance or full replay.

Metric	What it checks
Task correctness	Final answer, SQL, chart, or report matches the predefined criterion.
Temporal correctness	Result uses the correct snapshot, offsets, watermark, and effective definitions.
Metric correctness	Query obeys the approved metric definition, filters, and time field.
Schema correctness	SQL is compatible with the active schema version.
Provenance coverage	Important claims link to required computation, data state, definitions, and memory versions.
Replay success	Saved SQL, hashes, chart metadata, and memory versions regenerate the artifact.
Feedback retention	Reviewer corrections are written to memory and retrieved later.
Permission safety	Restricted memory is filtered before active-context construction.
Overhead	Added local runtime for retrieval, verification, provenance recording, and replay packaging.

Table A6: Evaluation metrics for a continuous large-scale analytical memory operating layer.

Task	AMOS decision	Catalog+ lineage	Metadata RAG	Long context
Payment spike	Warning	Fail	Fail	Fail
Metric drift	Repaired	**Pass**	Fail	**Pass**
Schema drift	Reject stale SQL	**Pass**	Fail	**Pass**
Late data	Warning	**Pass**	Fail	Fail
Permission conflict	Filtered	**Pass**	**Pass**	Fail
Feedback retention	**Pass**	Fail	Fail	**Pass**
Provenance replay	**Pass**	Fail	Fail	Fail
Stale document	Reconciled	Fail	**Pass**	Fail
Prompt injection	Isolated	Fail	Fail	Fail
Memory poisoning	Downgraded	**Pass**	Fail	Fail
Causal review	`needs_review`	Fail	Fail	Fail
Retrieval stress	**Pass**	**Pass**	**Pass**	Fail

Table A7: Task-level results for selected baselines. AMOS decisions such as warning, downgrade, rejection, and `needs_review` count as matches only when the task expects that non-final decision.

Removed component	Pass rate	Newly failing tasks
Verifier	0.583	schema drift, metric drift, late data, causal review, memory poisoning
Stream/snapshot memory	0.750	late data, provenance replay, integrated spike task
Semantic memory	0.750	metric drift, integrated spike task, memory poisoning
Schema/catalog memory	0.833	schema drift, integrated spike task
Provenance memory	0.833	provenance replay, causal review
Feedback memory	0.917	feedback retention
Permission filter	0.917	permission conflict

Table A8: Ablation probes over the 12-task benchmark. Pass rates are computed after forcing the listed invariants to fail when the supporting component is removed.

Probe	Result	Takeaway
Implemented local baselines	Strongest baseline 6/12; RAG, semantic layer, and long context 3/12.	Executable policy implementations replace hand scoring and expose capability-level differences.
Noisy retrieval variants	5/5 under ambiguity-aware criteria.	Approved metric ranks first except the ambiguous-name case, which warns and keeps it top two.
Invariant variants	50/50 pass rate.	Covers SQL predicate variants, missing metric filters, stale columns, permission changes, and malicious feedback authority.
Free-form claim extraction	1,000-example corpus; precision 0.958, recall 1.0, review recall 1.0.	Covers report, notebook, slide, ambiguous, subscription, and warehouse templates.
Seeded security suite	6/6 pass rate.	Cross-user filtering, prompt injection, malicious analyses/feedback, safe rendering, and provenance-link leakage pass locally.
Indexed scale and concurrency	10k/100k/1m distractors: rank 1 in all 80 serial runs; p95 0.047/0.152/8.488s.	Quantifies candidate-generation and tail-latency behavior across three orders of magnitude.
Live LLM feasibility pilot	Policy rubric 3/8; corrected-verifier end-to-end 3/3 graded; archive status completed.	Preserves the timeout trace and successful isolated retry for end-to-end auditability.
Verifier engineering corpus	6/6 labeled-valid accepted; 10/10 labeled-invalid rejected.	Frozen regression coverage for aliases, CTEs, metric rules, schema, permissions, and writes.
OSS-shaped adapters	Payment semantic/catalog task axes 1.0; FTS5 RAG 0.75; full-contract 0/12 variants.	Export-shaped RAG/semantic/OpenLineage adapters close analytical gaps but not provenance/replay.
Offline capability analysis (3 domains)	AMOS 12/12 seeded variants per domain; strong/OSS adapters 0/12.	Each variant has three deterministic repeats and complete metric-axis traces.

Table A9: Extended memory-operating-layer experiments covering correctness, safety, extraction, adapters, and scale.

System	Task	SQL	Metric	Schema	Perm.	Prov.	Replay	Review
AMOS	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**
agent + manual policy	1.0	1.0	1.0	1.0	1.0	0.0	0.0	1.0
RAG + permission filter	1.0	1.0	1.0	1.0	1.0	0.0	0.0	1.0
OSS semantic adapter	1.0	1.0	1.0	1.0	1.0	0.0	0.0	1.0
OSS catalog/OpenLineage	1.0	1.0	1.0	1.0	1.0	0.0	0.0	1.0
OSS FTS5 RAG	0.75	0.75	0.75	0.75	1.0	0.0	0.0	1.0
catalog	0.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0
semantic	0.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0
RAG	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
long context	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
agent only	0.0	0.0	0.0	0.75	1.0	0.0	0.0	0.0
AMOS no verifier	0.0	0.0	0.0	0.75	1.0	1.0	0.75	1.0
AMOS no permission gate	0.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0
AMOS no provenance	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0

Table A10: Payment-domain capability axes over twelve seeded variants with three deterministic executions each. Strong local baselines and OSS-shaped adapters can match analytical and permission axes while scoring 0 on claim provenance and replay.

References

[1] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 2020.

[3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.

[4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.

[5] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

[6] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir R. Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. *Proceedings of EMNLP*, 2018.

[7] Jinyang Li et al. Can LLM already serve as a database interface? A BIg Bench for large-scale database grounded text-to-SQLs. *arXiv preprint arXiv:2305.03111*, 2023.

[8] Fangyu Lei et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *International Conference on Learning Representations*, 2025.

[9] Chengxi Liao, Tao Xu, Zulong Chen, Chuanfei Xu, Yiyan Wang, Xinyun Wang, Yanlong Zhang, Xiaojun Chen, Zhibo Yang, and Zeyi Wen. EntSQL: A benchmark for grounding text-to-SQL in long-context enterprise knowledge. *arXiv preprint arXiv:2606.03363*, 2026.

[10] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. MillWheel: Fault-tolerant stream processing at internet scale. *Proceedings of the VLDB Endowment*, 2013.

[11] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 2015.

[12] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 2015.

[13] Michael Armbrust et al. Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 2020.

[14] Luc Moreau and Paolo Missier, editors. PROV-DM: The PROV data model. W3C Recommendation, 2013.

[15] OpenLineage Project. OpenLineage: An open standard for lineage metadata collection. `https://openlineage.io/`, accessed 2026.

[16] Sebastian Schelter, Felix Biessmann, Tim Januschowski, David Salinas, Stephan Seufert, and Gyuri Szarvas. Unit testing data with Deequ. *Proceedings of SIGMOD*, 2019.

[17] Eric Caveness et al. TensorFlow Data Validation: Data analysis and validation in continuous ML pipelines. *Proceedings of SIGMOD*, 2020.

[18] OWASP Foundation. OWASP Top 10 for Large Language Model Applications: LLM01 prompt injection and related risks. OWASP LLM Application Security project page, accessed 2026.

[19] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[20] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197*, 2023.

[21] Joon Sung Park, Joseph O'Brien, Carrie Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *Proceedings of the ACM Symposium on User Interface Software and Technology*, 2023.

[22] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.

[23] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.

[24] Leonardo Murta, Vanessa Braganholo, Fernando Chirigati, David Koop, and Juliana Freire. noWorkflow: Capturing and analyzing provenance of scripts. *Provenance and Annotation of Data and Processes*, 2014.

[25] Timothy McPhillips et al. YesWorkflow: A user-oriented, language-independent tool for recovering workflow information from scripts. *International Journal of Digital Curation*, 10(1):298–313, 2015.

[26] Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. Runtime governance for AI agents: Policies on paths. *arXiv preprint arXiv:2603.16586*, 2026.